

Notebook

February 8, 2026

1 Атака Винера

Расшифрование или создание подписи при помощи RSA - обычно достаточно энергоёмкий и долгий процесс, так как мы не можем просто выбрать красивую закрытую экспоненту d с низким весом Гэмминга (её очень просто было бы угадать). Поэтому у людей иногда появляется позыв выбрать такую открытую экспоненту e , что длина d в битах в несколько раз меньше, чем длина модуля N , например, N - 2048, а d - 500 бит. Интуитивно может показаться, что раз d сложно угадать (всё-таки 500 бит), можно быть спокойным за безопасность криптосистемы. К сожалению, это очень опасная ошибка.

1.1 Теорема (М. Wiener)

Пусть $N = pq$ с $q < p < 2q$. Пусть $d < \frac{1}{3}N^{\frac{1}{4}}$. Зная (N, e) , для которого $ed = 1 \bmod \varphi(N)$, Марвин может эффективно восстановить d .

1.1.1 Доказательство (взято из [Twenty Years of Attacks on the RSA Cryptosystem](#) в авторстве Dan Boneh)

Доказательство основывается на приближенных значениях при использовании непрерывных дробей. Поскольку существует $ed = 1 \bmod \varphi(N)$, то существует такое k , что $ed - k\varphi(N) = 1$. Следовательно,

$$\left| \frac{e}{\varphi(N)} - \frac{k}{d} \right| = \frac{1}{d\varphi(N)}$$

Значит, $\frac{k}{d}$ - это приближенное значение к $\frac{e}{\varphi(N)}$. Хотя Марвин и не знает $\varphi(N)$, он может воспользоваться N , чтобы получить его примерную оценку. Действительно, раз $\varphi(N) = N - p - q + 1$ и $p + q - 1 < 3\sqrt{N}$, получается, что $|N - \varphi(N)| < 3\sqrt{N}$

Подставляя N вместо $\varphi(N)$ получаем:

$$\left| \frac{e}{N} - \frac{k}{d} \right| = \left| \frac{ed - k\varphi(N) - kN + k\varphi(N)}{Nd} \right| = \left| \frac{1 - k(N - \varphi(N))}{Nd} \right| \leq \left| \frac{3k\sqrt{N}}{Nd} \right| = \frac{3k}{d\sqrt{N}}$$

Заметим, что $k\varphi(N) = ed - 1 < ed$. Так как $e < N$, видно, что $k < d < \frac{1}{3}N^{\frac{1}{4}}$. Поэтому получаем:

$$\left| \frac{e}{N} - \frac{k}{d} \right| \leq \frac{1}{dN^{\frac{1}{4}}} < \frac{1}{2d^2}$$

Это классическое отношение приближения. Количество дробей $\frac{k}{d}$ with $d < N$, дающих приближенное значение $\frac{e}{N}$ с такой точностью, ограничено $\log_2 N$. На самом деле, все такие дроби могут быть получены как подходящие дроби непрерывной дроби $\frac{e}{N}$. Всё, что надо сделать, это посчитать $\log N$ подходящих дробей непрерывной дроби $\frac{e}{N}$. Одна из них будет равна $\frac{k}{d}$. Так как $ed - k\varphi(N) = 1$, то $(k, d) = 1$ и значит $\frac{k}{d}$ - это сокращенная дробь. Это и есть линейный по времени алгоритм восстановления закрытого ключа d . **ЧТД**

1.2 Непрерывные и подходящие дроби

Непрерывная дробь - это выражение, полученное при помощи последовательного процесса представления числа как суммы его целочисленной части и обратного элемента по умножению к оставшейся части, далее представляя эту оставшуюся часть, как сумму целочисленной части и обратного элемента и так далее. Пример: Для действительного числа $x > 0$ и целых положительных a_i , for $i = 1, \dots, n$

$$x = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{\ddots + \frac{1}{a_n}}}}$$

- это непрерывная дробь. Целые числа a_0, a_1 , и т.д., называются *элементами* или *неполными частными* непрерывной дроби. Есть много способов записи непрерывных дробей, мы будем использовать следующую:

$$x = [a_0; a_1, a_2, \dots]$$

Непрерывная дробь может сформировать своё приближение при помощи первых элементов. Такие приближения называются *подходящими дробями*. Очевидно, что для рационального x количество таких подходящих дробей конечно. Первые четыре подходящие дроби непрерывной дроби:

$$\frac{a_0}{1}, \frac{a_1 a_0 + 1}{a_1}, \frac{a_2(a_1 a_0 + 1) + a_0}{a_2 a_1 + 1}, \frac{a_3(a_2(a_1 a_0 + 1) + a_0) + (a_1 a_0 + 1)}{a_3(a_2 a_1 + 1) + a_1}$$

Если существуют последующие подходящие дроби, они могут быть вычислены рекурсивно. h_i обозначим числители, а k_i - знаменатели. Тогда ряд подходящих дробей может быть вычислен следующим образом:

$$\frac{h_i}{k_i} = \frac{a_i h_{i-1} + h_{i-2}}{a_i k_{i-1} + k_{i-2}}$$

Реализуйте алгоритм для нахождения элементов и подходящих дробей для любого рационального числа.

Подсказка: вы можете использовать алгоритм Евклида для вычисления элементов

[]:

Теперь, когда Вы написали алгоритм поиска подходящих дробей, давайте рассмотрим, как с их помощью найти p и q . Поскольку $\frac{k}{d}$ - это сокращенная дробь,

знаменатель одной из подходящих дробей - это и есть закрытая экспонента d . Этого уже достаточно, чтобы расшифровать шифртекст, но мы можем и факторизовать N . Мы также знаем k , а $\varphi(N) = \frac{ed-1}{k}$, значит мы можем вычислить $\varphi(N)$.

$$N - \varphi(N) = pq - (p-1)(q-1) = pq - pq + p + q - 1 = p + q - 1$$

$$q = N - \varphi(N) - p + 1$$

$$N = pq = p(N - \varphi(N) - p + 1) = pN - p\varphi(N) - p^2 + p$$

$$p^2 - p(N - \varphi(N) + 1) + N = 0$$

Получаем красивое квадратное уравнение, корнями которого и будут p и q

Давайте используем полученные знания на уязвимом сервере. Задача такая же, как и в blinding таске. Нужно отправить на сервер правильную подпись для сообщения 'flag', чтобы получить флаг. Шаги:

1. Найти d , хитроумно используя непрерывные дроби
2. Найти p и q
3. Подписать сообщение и послать на сервер
4. Profit

```
[ ]: import socket
import re
from Crypto.Util.number import inverse, long_to_bytes, bytes_to_long
class VulnServerClient:
    def __init__(self, show=True):
        """Инициализация, подключаемся к серверу"""
        self.s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone', 1338))
        if show:
            print (self.recv_until().decode())
    def recv_until(self, symb=b'\n>'):
        """Получение сообщения с сервера, по умолчанию до приглашения_
↪К вводу команды"""
        data=b''
        while True:
            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data
    def get_public_key(self, show=True):
        """Получение открытого ключа с сервера"""
        self.s.sendall('public\n'.encode())
        response=self.recv_until().decode()
        if show:
            print (response)
        e=int(re.search(r'(?<=e: )\d+', response).group(0))
```

```

N=int(re.search(r'(?<=N: )\d+',response).group(0))
self.num_len=len(long_to_bytes(N))
self.e,self.N=e,N
return (e,N)

def checkSignatureNumber(self,c,show=True):
    """Проверка сигнатуры (на сервере) для подписи в числовом_
↪представлении"""
    try:
        num_len=self.num_len
    except AttributeError:
        print ('You need to get the public key from the server_
↪first')
    return
    signature_bytes=long_to_bytes(c,num_len)
    self.checkSignatureBytes(signature_bytes,show)

def checkSignatureBytes(self,c,show=True):
    """Проверка сигнатуры (на сервере) для подписи в байтовом_
↪представлении"""
    try:
        num_len=self.num_len
    except AttributeError:
        print ('You need to get the public key from the server_
↪first')
    return
    if len(c)>num_len:
        print ("The message is too long")
        return

    hex_c=c.hex().encode()
    self.s.sendall(b'flag '+hex_c+b'\n',)
    response=self.recv_until(b'\n').decode()

    if show:
        print (response)

    if response.find('Wrong')!=-1:
        print('Wrong signature')
        x=self.recv_until()
        if show:
            print (x)
        return
    flag=re.search(r'CRYPTOTRAINING\{.*\}',response).group(0)
    print ('FLAG: ',flag)
def setPrivateKey(self,p,q):
    """Выставить закрытый ключ"""

```

```

        self.p=p
        self.q=q
        self.d=inverse(self.e,(p-1)*(q-1))

    def signMessageBytes(self,m):
        """Подписать сообщение, после того как найден закрытый ключ"""
        try:
            num_len=self.num_len
        except AttributeError:
            print ('You need to get the public key from the server_
↪first')
            return
        if len(m)>num_len:
            print('m too long')
        if len(m)<num_len:
            m=bytes([0x0]*(num_len-len(m))) + m
        signature_bytes=long_to_bytes(pow(bytes_to_long(m),self.
↪d,self.N))
        return signature_bytes

    def __del__(self):
        self.s.close()

```

```

[ ]: vs=VulnServerClient()
    (e,N)=vs.get_public_key()

```

```

[ ]:

```